GateRank: Long-Context Enhancement of Linear Attention via Gated Fast and Slow Dynamics

Constantin Kronbichler
Forschungszentrum Jülich
ckronbichler@proton.me

Jan Finkbeiner
Forschungszentrum Jülich
RWTH Aachen University
j.finkbeiner@fz-juelich.de

Emre Neftci
Forschungszentrum Jülich
RWTH Aachen University

Abstract

Long-context sequence modeling demands mechanisms that retain and selectively access information across diverse timescales. We introduce GateRank, a dual timescale extension to linear attention that maintains fixed-size fast and slow memory states, with an input-dependent gate to consolidate information into the slow state. Thereby, GateRank relaxes the 1-semiseparable decay constraint of linear attention models while preserving streaming inference and chunk-parallel training. Integrated with Mamba2 and Gated DeltaNet, GateRank achieves superior performance on synthetic associative recall tasks thanks to the more expressive memory dynamics operating on two timescales. We train 760M-parameter GateRank models that match or slightly improve standard language modeling results and yield significant gains on retrieval-heavy tasks such as Needle-in-a-Haystack and LongBench. These findings suggest that multi-timescale memory can extend the expressive capacity of linear-time sequence models without sacrificing efficiency. Code available here: https://anonymous.4open.science/r/gaterank-746E

1 Introduction

Scaling the self-attention based transformer has led to remarkable performances on language tasks [34, 6]. However, extending the context of language models requires processing longer token sequences, which leads to decreased throughput due to the quadratic compute complexity and linear memory complexity of softmax-based self-attention with respect to sequence length. State space models (SSMs) and linear attention models address this issue by using a fixed size memory. This is achieved by dropping the softmax non-linearity and accumulating outer products of keys and values into the hidden state instead of explicitly keeping a list of all past key-value pairs [18]. While linear attention improves throughput, the fixed size memory impacts language modeling and information retrieval performance [1]. Subsequently, many works explored modifications to the linear attention mechanism to mitigate the loss in modeling performance while maintaining the throughput advantage. Works such as Mamba2 [11], GLA [36], or RetNet [32] have introduced gating mechanisms to the recurrence of the hidden state, leading to linear attention models that achieved comparable results to softmax-based attention in most language modeling settings.

Yet, linear attention models lag behind softmax-based attention on associative recall across long contexts [1]. DeltaNet partly addresses this by orthogonalizing hidden state updates to maximize retention [37], and Gated DeltaNet combines this with Mamba2-style input-dependent gating to achieve state-of-the-art linear attention performance [38]. However, a structural limitation persists: the decay mask that is applied to the recurrent hidden state in these models is rank-1 semi-separable. This means that a single decay schedule is applied to all past tokens. As a result, the model can tune how quickly memory fades, but cannot retain some content while letting other content decay, a flexibility softmax attention has by default, since it imposes no global decay over keys.

Preprint.

GateRank relaxes this 1-semiseparable decay constraint by introducing a learnable rank structure via a ReLU-activated signal that acts both as a reset of local decay dynamics and as a global decay. The dual state update scheme is motivated by the same continuous-time perspective underlying SSM and linear attention recurrences. However, while previous mechanisms use a single timescale discretization, GateRank uses a dual timescale discretization leading to the dual state update scheme. Hence, GateRank can be used to augment any linear attention model with decay masking, such as Mamba2 or Gated DeltaNet. In auto-regressive predictions this results in slow and fast dynamics on the hidden state. Non-zero decays predicted for the slow state decay indicate a slow state update and result in a reset of the fast state.

We provide Triton kernels [33] for the GateRank extension with Mamba2 and Gated DeltaNet as base models. Using integer masking we achieve an efficient and scalable chunk scan implementation that surpasses flash attention throughput for longer sequences enabling chunk-parallel training and streaming auto-regressive inference. GateRank achieves clear improvements in associative recall performance as measured on the synthetic Multi-Query Associative Recall (MQAR) [1] task. At the 760M-parameter scale GateRank models marginally improve in standard language benchmarks. However, there is a clear advantage in tasks that require advanced memory management over long contexts, such as Needle-In-A-Haystack evaluations [16] or LongBench-E [3].

Overall GateRank presents a novel tradeoff between compute requirements and long-context associative recall capabilities that significantly improves over the linear attention baseline while maintaining the same linear asymptotic complexity and avoiding the quadratic complexity penalty of self-attention.

2 Related work

Replicating softmax' low-entropy query-key interactions One advantage of softmax-attention over linear attention models is the low-entropy attention matrix, i.e. softmax $\left(QK^T/\sqrt{d_k}\right)$, meaning that only very few query-key relations become very large while most of them stay very low [40]. A complementary line of work sharpens linear attention's high-entropy attention matrix via kernel feature maps that approximate softmax [7, 40, 2, 42]; GateRank instead reduces entropy by breaking the monotonicity of the decay.

Gating in linear attention / RNNs / state-space models (SSMs) Early linear RNNs (S4, S5, LRU, RWKV 4/5, RetNet) used input-independent gating [14, 31, 23, 25, 32]. Mamba [13] introduced input-dependent gating, and subsequent works (Mamba2 [11], GLA [36], RWKV 6 [26]) simplified the decay for matmul-friendly chunked training, increasing usable state size. However, outer-product accumulation can still overwrite state. The delta rule [29, 37] addresses this by orthogonalizing updates, and Gated DeltaNet [38] combines it with Mamba2-style gating. Related rules include RWKV 7 [27] and Longhorn [19].

Higher-rank state transitions The temporal rank, which governs the complexity of interactions across the sequence length, is different to the state rank, which determines the expressivity of cross-channel mixing within the transition matrix. This distinction is formalized through the tensor-centric view of state transitions introduced in [15], which separates the dimensionality of the hidden state from the rank of its update rule. Models like DeltaNet and Gated DeltaNet [37, 38] utilize a diagonal plus rank-1 structure, corresponding to a single step of online gradient descent. To enhance state-tracking capabilities, DeltaProduct [30] transitions to a diagonal plus rank-N structure by utilizing a product of N generalized Householder transformations. This trajectory towards higher expressivity is unified by the Fixed-Point RNN framework, which demonstrates that dense linear transitions can be recovered as the fixed point of such iterative diagonal updates [22].

Unstructured masking for linear attention models With log-linear attention there has been a concurrent attempt to enhance linear attention models by breaking up the monotone structure of the decay mask [15]. While GateRank's state-size is still fixed with respect to the sequence length, albeit doubled with respect to the baseline linear attention model, log-linear attention uses a pre-defined split into decay segments that leads to logarithmic memory growth during auto-regressive inference. The main difference between GateRank and log-linear attention is the learnable location of decay segment boundaries in GateRank.

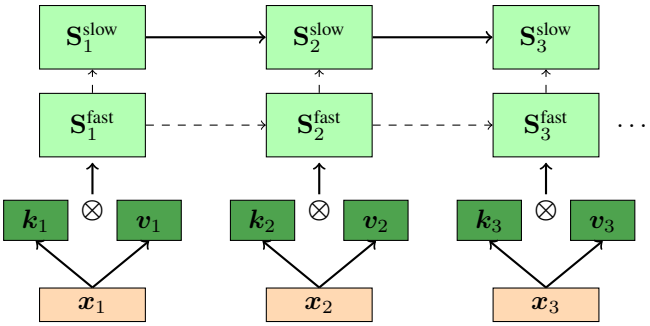

Figure 1: Recurrent mode for GateRank. Dashed arrows indicate a gating mechanism that does not necessarily propagate the value along this arrow at every timestep, depending on g_{slow}.

3 GateRank

GateRank provides a more general gating mechanism compared to the one introduced by Mamba2 [11] which is also used in other linear attention models such as Gated DeltaNet [38]. For simplicity, we describe the recurrent, parallel, and chunkwise parallel form only for the GateRank + Mamba2 extension but note that it applies to all other linear attention models with input-conditioned gating such as Gated DeltaNet [38], as demonstrated by our implementation for both Mamba2 and Gated DeltaNet. GateRank splits the Mamba2-like gated state update into a hierarchy of two states: a state with fast updates that accumulates key-value outer products at every timestep, just like the hidden state in Mamba2, and, a slow state that intermittently accumulates the fast state. Whenever the fast state is added to the slow state, the fast state is reset to zero.

State-space equations The discrete-time state-space equations are based on two states with different decays α_t and β_t:

$$\mathbf{S}_t^{\text{slow}} = \alpha_t \mathbf{S}_{t-1}^{\text{slow}} + \mathbb{1}\left(g_{\text{slow}} < 0\right) \alpha_t \beta_t \mathbf{S}_{t-1}^{\text{fast}} \tag{1a}$$

$$\mathbf{S}_t^{\text{fast}} = \mathbb{1}\left(g_{\text{slow}} = 0\right) \beta_t \mathbf{S}_{t-1}^{\text{fast}} + \boldsymbol{k}_t \boldsymbol{v}_t^T \tag{1b}$$

$$\boldsymbol{o}_t = \boldsymbol{q}_t^T (\mathbf{S}_t^{\text{slow}} + \mathbf{S}_t^{\text{fast}}) \tag{1c}$$

where $\mathbb{1}(\text{cond}) = \begin{cases} 1 \text{ if cond} \\ 0 \text{ otherwise} \end{cases}$.

Figure 1 shows an illustration of the recurrent form of GateRank. Note that the gating mechanism of Equation 1 is a more general form of the Mamba2 gating mechanism: if the slow state remains untouched, i.e. $g_{\text{slow}} = 0$, we recover the Mamba2 discrete-time state-space equations.

Gate computation Both gates are computed in log-space. The slow state gate g_{slow} is computed via a projection followed by a short convolution of length 4 and a ReLU activation:

$$g_{\text{slow}} = \log(\alpha_t) = -A_{\text{slow}} \,\text{ReLU}\left(\text{ShortConv}\left(W_\alpha x_t + b_\alpha\right)\right) \tag{2}$$

where A_{slow} is a scalar decay coefficient, learned via gradient descent. We constrain the learned scalar to be strictly positive ($A_{\text{slow}} > 0$). Since ReLU outputs are non-negative, the gate values are bounded such that $g_{\text{slow}} \leq 0$, which ensures the decay factor $\alpha_t = \exp(g_{\text{slow}})$ remains strictly within $(0, 1]$.

Using ReLU (rather than softplus) lets g_{slow} output exactly zero, so $\alpha_t = 1$ corresponds to no decay, enabling sparse, learnable retention periods; $g_{\text{slow}} < 0$ marks discrete reset segment boundaries where the slow state accumulates the fast state and the fast state is reset to zero.

The fast decay β_t is computed just like the decay in Mamba2 via `softplus`:

$$g_{\text{fast}} = \log(\beta_t) = -A_{\text{fast}}\text{softplus}(W_\beta x_t + b_\beta) \tag{3}$$

ensuring continuous, non-zero decay within reset segments.

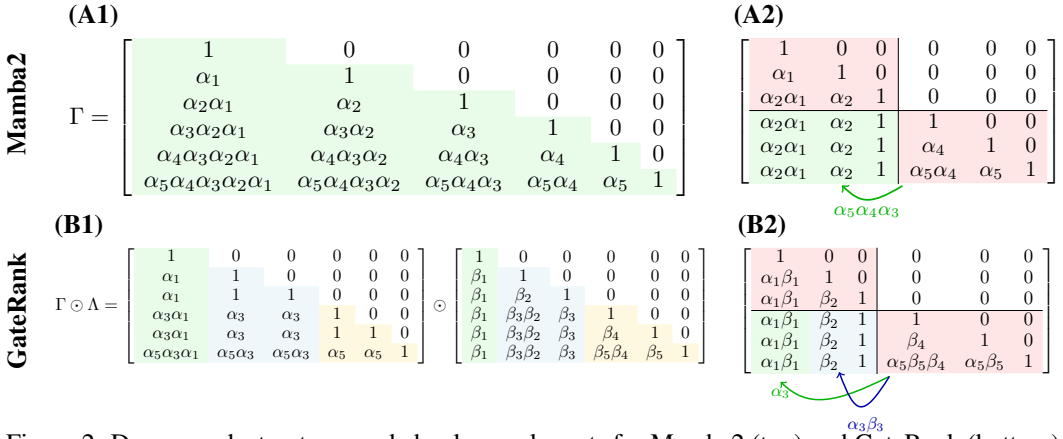

Figure 2: Decay mask structures and chunk-scan layouts for Mamba2 (top) and GateRank (bottom). **(A1)** Mamba2's parallel-form mask is rank-1 semi-separable: $\Gamma_{i,j} = \gamma_i/\gamma_j$ with $\gamma_i = \prod_{j=1}^{i} \alpha_j$, applying the same decay schedule across all positions. **(A2)** Mamba2's chunk scan: the inter-chunk transfer only requires a single materialized state at the chunk boundary, since every column on a given row shares the same gate. **(B1)** GateRank's parallel-form mask $\Gamma \odot \Lambda$ has slow-state decays only at reset boundaries (here $t \in \{1, 3, 5\}$, where $g_{\text{slow}} < 0$); elsewhere $\alpha_t = 1$. Within each reset segment, β gates form local cumulative products, yielding a higher-rank structure. **(B2)** GateRank's chunk scan materializes both states at the boundary: completed reset segments (green) feed the slow state, while the ongoing segment (blue) feeds the fast state, each requiring a different inter-chunk decay.

Motivation via dual timescale discretization of continuous-time system We consider the same continuous-time system as used to derive the Mamba2 model updates [11] parameterized by a decay matrix $\boldsymbol{A}$, an input projection $\boldsymbol{B}(t)$, and an output projection $\boldsymbol{C}(t)$, driven by the input signal $\boldsymbol{v}(t)$:

$$\dot{\mathbf{S}}(t) = \boldsymbol{A}\mathbf{S}(t) + \boldsymbol{B}(t)\boldsymbol{v}(t)^\top \qquad \boldsymbol{y}(t) = \boldsymbol{C}(t)\mathbf{S}(t) \tag{4}$$

To capture the multi-timescale dynamics, we introduce two distinct discretization timescales: a fast timescale Δ_t^{fast} corresponding to the regular input sampling rate, and an input-dependent slow timescale Δ_t^{slow} representing intermittent memory consolidation events (where $\Delta_t^{\text{slow}} \geq \Delta_t^{\text{fast}}$). Applying a zero-order hold (ZOH) assumption, we define the discretized gate variables as the exact discrete-time transitions of $-\boldsymbol{A}_{\text{fast}}$ and $-\boldsymbol{A}_{\text{slow}}$ over their respective timescales:

$$\alpha_t = \exp(-\boldsymbol{A}_{\text{slow}}\Delta_t^{\text{slow}}) \qquad \beta_t = \exp(-\boldsymbol{A}_{\text{fast}}\Delta_t^{\text{fast}}) \tag{5}$$

with $\Delta_t^{\text{slow}} = \text{ReLU}(\text{ShortConv}(W_\alpha x_t + b_\alpha))$ and $\Delta_t^{\text{fast}} = \text{softplus}(W_\beta x_t + b_\beta)$. Similarly, the discrete-time keys $\boldsymbol{k}_t$ are obtained by integrating $\boldsymbol{B}(t)$ over the fast timescale, yielding:

$$\boldsymbol{k}_t = -\boldsymbol{A}_{\text{fast}}^{-1}(\exp(-\boldsymbol{A}_{\text{fast}}\Delta_t^{\text{fast}}) - \boldsymbol{I})\boldsymbol{B}_t \tag{6}$$

The discrete-time queries map directly to the output projection, $\boldsymbol{q}_t = \boldsymbol{C}_t$. Integrating over these asynchronous intervals decomposes the continuous dynamics into local fast accumulations between slow boundaries plus a transfer of the fast accumulator into the slow buffer at each boundary. Equation 1 is slightly altered from the resulting dual-timescale ZOH discretization: applying the slow decay α_t after the fast state is added allows for a simpler implementation, because the decay matrix Γ from Figure 2 (B1) retains the standard linear-attention form $\Gamma_{i,j} = \gamma_i/\gamma_j$ and decay accumulation can reuse cumulative-product computations without extra indexing operations (see below for parallel and chunk scan formulations). Since A_{slow} and A_{fast} are learned independently, the resulting extra α_t factor on the consolidated chunk is absorbable into the learned decays, leaving the model class unchanged. Further details, including the strict ZOH form, are provided in Appendix B.

Parallel form The parallel form multiplies by two mask matrices Λ and Γ as shown in Figure 2 (B1):

$$\mathbf{O} = (\mathbf{Q}\mathbf{K}^T \odot \Gamma \odot \Lambda \odot \mathbf{M})\mathrm{V} \tag{7}$$

where $\Gamma_{i,j} = \frac{\gamma_i}{\gamma_j}$ using $\gamma_i = \prod_{j=1}^{i} \alpha_j$ as in Mamba2, and $\mathbf{M}$ is the causal mask. Figure 2 (B1) shows an example with resets at positions $\{1, 3, 5\}$ creating three reset segments colored distinctly:

positions 1 (green), positions 2-3 (blue), and positions 4-6 (yellow). Within each reset segment, β gates create local cumulative products. For a comparison with the 1-SS mask of Mamba2 that omits the Λ mask see Figure 2 (A1).

To define Λ, we introduce $r_{\text{next}}(i, l)$ which returns the start of the next reset segment after position i up to position l:

$$r_{\text{next}}(i,l) = \begin{cases} \min\{t \in (i,l] \mid g_{\text{slow},t} < 0\} & \text{if such } t \text{ exists} \\ l+1 & \text{otherwise} \end{cases} \tag{8}$$

The local cumulative product within each reset segment is:

$$\lambda_i = \prod_{j=s_i}^{i} \beta_j \qquad s_i = \begin{cases} 1 + \max\{t < i \mid g_{\text{slow},t} < 0\} & \text{if such } t \text{ exists} \\ 1 & \text{otherwise} \end{cases}$$

where s_i just denotes the start of the reset segment that contains position i. Using these definitions, the decay mask Λ calculates the cumulative decay in the current reset segment or up to the diagonal, i.e. index i, if there is no reset before it:

$$\Lambda_{i,j} = \frac{\lambda_{\min(r_{\text{next}}(j,i),i)}}{\lambda_j}. \tag{9}$$

As visible in Figure 2 (B1), this creates higher-rank structure: each reset segment forms a rank-1 block, but multiple segments yield rank greater than 1 overall.

Chunk recurrent form For efficient computation, GateRank uses a chunk scan implementation (see Figure 2 (B2)) similar to the implementation of Mamba2 [11] and flash linear attention [36] (see Figure 2 (A2)). Using the chunk notation from [38], we denote $\mathbf{Q}_{[t]} := \boldsymbol{q}_{tC:(t+1)C}$ as the query block for chunk t with chunk size C, and $\boldsymbol{q}^j_{[t]} := \boldsymbol{q}_{tC+j}$ as the j-th query within chunk t where $j \in \{0, 1, ..., C-1\}$. $\lambda^j_{[t]}$ denotes λ_{tC+j}, the cumulative product at position j within chunk t.

The chunk scan must distinguish key-value pairs in completed reset segments from those in the ongoing final segment. We do this by materializing both the slow state $S^{\text{slow}}_{[t]}$ and fast state $S^{\text{fast}}_{[t]}$ at the chunk boundary: completed segments contribute to the slow state, the ongoing segment to the fast state. Figure 2 (B2) illustrates this: the green column shows the single decay to the slow state, while the blue columns show the different decays for the fast state's ongoing reset segment. In contrast, Mamba2, which lacks slow state updates, only needs to track a single state at the chunk boundary, as illustrated in Figure 2 (A2).

The other difference from Mamba2's chunk scan is that decays are computed over reset segments rather than the full sequence. This affects two places where decays are applied. First, when computing a chunk's contribution to the state (e.g., the bottom-left chunk of Figure 2 (B2)), each key-value pair needs the decay $\boldsymbol{d}_{\text{last}}$ from its column to the last column of the chunk. Second, when reading out the chunk-boundary states with the query (e.g., the bottom-right chunk of Figure 2 (B2)), each query (row j in the decay matrix) needs the decay $\boldsymbol{d}_{\text{first}}$ from its position back to the first token of the chunk:

$$\boldsymbol{d}^j_{\text{last},[t]} = \frac{\lambda^{\min(r_{\text{next}}(j,C),C-1)}_{[t]}}{\lambda^j_{[t]}} \qquad \boldsymbol{d}^j_{\text{first},[t]} = \lambda^{r_{\text{next}}(-1,j)}_{[t]}$$

where $\min\left(r_{\text{next}}(j, C), C-1\right)$ is the next reset position after j within the chunk or the last position $C-1$ in the chunk if it does not contain another reset past position j. With the notation from [38], the chunk-level recurrence becomes:

$$\mathbf{S}^{\text{slow}}_{[t+1]} = \overrightarrow{\mathbf{S}}^{\text{slow}}_{[t]} + b_{[t]} \overrightarrow{\mathbf{S}}^{\text{fast}}_{[t]} + \mathbf{V}^{\mathsf{T}}_{[t]} \overrightarrow{\mathbf{K}}^{\text{slow}}_{[t]} \qquad \mathbf{S}^{\text{fast}}_{[t+1]} = (1 - b_{[t]}) \overrightarrow{\mathbf{S}}^{\text{fast}}_{[t]} + \mathbf{V}^{\mathsf{T}}_{[t]} \overrightarrow{\mathbf{K}}^{\text{fast}}_{[t]} \tag{10a}$$

$$\mathbf{O}_{[t]} = \left(\overleftarrow{\mathbf{Q}}^{\text{slow}}_{[t]} \mathbf{S}^{\text{slow}\,\mathsf{T}}_{[t]} + \overleftarrow{\mathbf{Q}}^{\text{fast}}_{[t]} \mathbf{S}^{\text{fast}\,\mathsf{T}}_{[t]} \right) + \left(\mathbf{Q}_{[t]} \mathbf{K}^{\mathsf{T}}_{[t]} \odot \Gamma_{[t]} \odot \Lambda_{[t]} \odot \mathbf{M} \right) \mathbf{V}_{[t]} \tag{10b}$$

using $b_{[t]}$ as a boolean flag: $b_{[t]} = 1$ if chunk t contains any position with $g_{\text{slow},tC+j} < 0$, and $b_{[t]} = 0$ otherwise. This flag determines whether the fast state gets consolidated into the slow state at the chunk boundary. The arrows denote decay-weighted quantities. The blue terms track the fast state (ongoing reset segment) while green terms track the slow state (completed reset segments).

The asymmetry in the chunk boundary update arises because key-value pairs from completed reset segments accumulate in the slow state, while pairs from the final ongoing segment accumulate in the fast state.

We define the *last reset segment* in chunk t as all positions r such that no reset occurs between r and the chunk end C. Formally, the last reset segment is defined as $\{j \mid j \in [0, C-1], r_{\text{next}}(j, C-1) = C\}$. This determines which keys contribute to each state:

$$
\begin{aligned}
\overleftarrow{\boldsymbol{q}}^{\text{slow},j}_{[t]} &= \gamma^{j}_{[t]} \boldsymbol{q}^{j}_{[t]} \qquad \overleftarrow{\boldsymbol{q}}^{\text{fast},j}_{[t]} = \gamma^{j}_{[t]} \boldsymbol{d}^{j}_{\text{first},[t]} \boldsymbol{q}^{j}_{[t]} \\
\overrightarrow{\boldsymbol{k}}^{\text{slow},j}_{[t]} &= \begin{cases} \frac{\gamma^{C-1}_{[t]}}{\gamma^{j}_{[t]}} \boldsymbol{d}^{j}_{\text{last},[t]} \boldsymbol{k}^{j}_{[t]} , & \text{if } j \textbf{ not} \text{ in last reset segment} \\ 0 , & \text{otherwise} \end{cases} \\
\overrightarrow{\boldsymbol{k}}^{\text{fast},j}_{[t]} &= \begin{cases} \frac{\gamma^{C-1}_{[t]}}{\gamma^{j}_{[t]}} \boldsymbol{d}^{j}_{\text{last},[t]} \boldsymbol{k}^{j}_{[t]} , & \text{if } j \text{ in last reset segment} \\ 0 , & \text{otherwise} \end{cases} \\
\overrightarrow{\mathbf{S}}^{\text{slow}}_{[t]} &= \gamma^{C-1}_{[t]} \mathbf{S}^{\text{slow}}_{[t]} \qquad \overrightarrow{\mathbf{S}}^{\text{fast}}_{[t]} = \gamma^{C-1}_{[t]} \boldsymbol{d}^{C-1}_{\text{first},[t]} \mathbf{S}^{\text{fast}}_{[t]}
\end{aligned} \tag{11}
$$

See appendix A on how the implementation uses integer masking for efficient computation.

4 Experimental Setup

4.1 MQAR: Associative Recall Analysis

We use the Multi-Query Associative Recall (MQAR) task [1], in which sequences of key-value pairs are followed by queries the model must answer with the matching value. All MQAR experiments use two-layer models with $d_{\text{model}} = 64$, vocabulary size 8192, and sweep 4 learning rates across 4 seeds. We run three complementary tests; full configurations are in Appendix C.

Standard MQAR We compare GateRank against Mamba2, Gated DeltaNet, and a Transformer at sequence lengths 512 and 1024 with 64 and 128 key-value pairs respectively, probing recall under increasing memory load. Each association is encoded as a (key, value) pair, so the model must store one new fact every two tokens.

Compositional MQAR with frozen resets Compositional MQAR is a two-hop variant in which each association is encoded as a (key_1, key_2, value) triple: the model must first retrieve the intermediate key_2 from key_1, then use key_2 to retrieve the final value. The storage rate is one fact every three tokens, rather than every two as in standard MQAR. The lower storage density leaves more slack between memory events, which lets us test a slightly wider range of reset frequencies and makes the gap between dense, alternating, and every-third resets more visible. We hardcode g_{slow} to fire at every token, every second token, or every third token (with a small fixed value 0.01) and compare against the learned reset gate.

State size scaling GateRank effectively doubles the recurrent state by maintaining a fast and a slow state. To verify that the gain comes from the higher-rank decay structure rather than from this doubled capacity, we sweep the number of key-value pairs and compare GateRank + Mamba2 (size 128 per state) against Mamba2 at the matched per-state size (128) and at the matched total state size (256). To make the differences between the curves clearly visible already at the shorter sequence length and keep the runtime of this larger sweep manageable, we use a slightly downscaled configuration relative to Figure 3, which explains the slightly worse results (see Appendix C for details).

4.2 Language Modeling and Retrieval Evaluation

We train 760M-parameter models on 30B tokens from FineWeb-Edu [17] with sequence length 4096. Models use dimension 1536, with Mamba2-based models using 20 layers and Gated DeltaNet-based models using 18 layers. The Transformer reference model is parameter matched resulting in 24 layers at model dimension 1536. We evaluate language modeling via perplexity on WikiText [20] and LAMBADA [24], and common sense reasoning using the EleutherAI evaluation harness [12, 4]. For

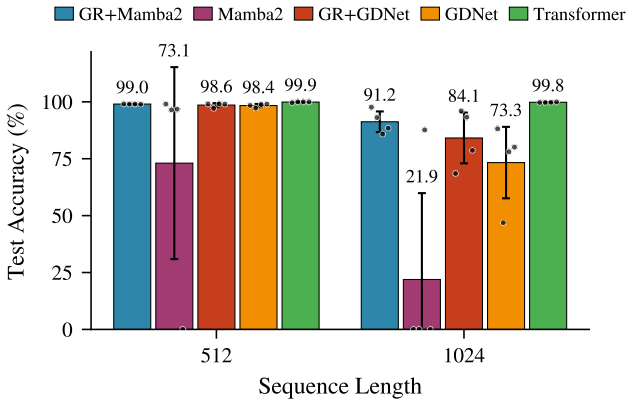

Figure 3: MQAR test accuracy at sequence lengths 512 (kv=64) and 1024 (kv=128). GateRank substantially improves both linear-attention baselines, especially at the longer sequence length where Mamba2 collapses.

Table 1: Per-layer firing ratio of the slow gate ($g_{\text{slow}} < 0$) for GateRank + Mamba2, averaged across heads and 4 seeds. Resets become sparser at longer sequence length.

Layer	Seq. Len. 512	Seq. Len. 1024
Layer 0	0.2981	0.1600
Layer 1	0.3279	0.2319
Overall	0.3130	0.1960

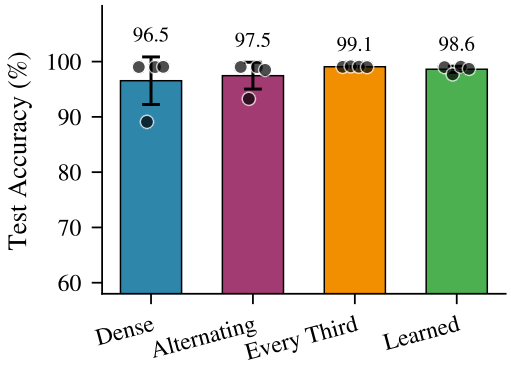

Figure 4: Compositional MQAR accuracy with hardcoded resets at every position (Dense), every second (Alternating), and every third (Every Third), versus the learned reset gate. Sparser resets improve recall, and the learned schedule approaches the best hardcoded one.

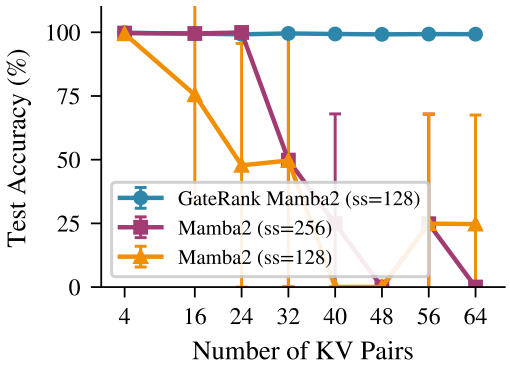

Figure 5: State size scaling on MQAR at sequence length 512. GateRank + Mamba2 (state size 128) outperforms Mamba2 at both the matched per-state size (128) and the matched total state size (256), indicating the gain comes from the higher-rank decay rather than raw capacity.

associative recall, we use the Structured Needle-In-A-Haystack (S-NIAH) benchmark from RULER [16] at context lengths of 1K, 2K, and 4K tokens. Lastly, LongBench-E [3] is used to evaluate the models on more realistic long-context tasks at 4K token sequence length. Training details and benchmark configurations are provided in Section D.

4.3 Throughput Evaluation

We benchmark the naive Triton chunk scan implementation by measuring the kernel's forward and backward time across sequence lengths from short contexts to 64K tokens, comparing GateRank against Mamba2, Gated DeltaNet, and FlashAttention-v2 [10].

5 Results and Discussion

5.1 MQAR: Associative Recall Analysis

Figure 3 reports MQAR accuracy across 4 seeds. Only the Transformer reaches 99%+ at both sequence lengths, while baseline Mamba2 collapses at length 1024. GateRank substantially improves

Table 2: Language modeling perplexity and zero-shot common sense reasoning for 760M models.

Model	**Wiki.** ppl ↓	**LMB.** ppl ↓	**LMB.** acc ↑	**PIQA** acc ↑	**Hella.** acc_n ↑	**Wino.** acc ↑	**ARC-e** acc ↑	**ARC-c** acc_n ↑	**OBQA** acc_n ↑	**SCIQ** acc_n ↑	**BoolQ** acc ↑	**Avg.** ↑
Transformer	19.98	17.51	42.17	68.82	50.66	56.83	66.29	33.02	36.60	81.40	57.37	54.80
Mamba2	21.13	19.65	39.59	**71.00**	**51.17**	**56.83**	66.50	33.36	**39.20**	77.50	56.85	54.67
GateRank+Mamba2	20.80	19.98	39.12	70.29	50.57	54.70	67.30	33.96	36.80	81.20	**59.88**	54.87
GDNet	**20.79**	17.53	40.71	70.62	50.91	55.41	67.76	**36.01**	36.60	81.60	56.12	55.08
GateRank+GDNet	21.12	**16.29**	**43.06**	70.95	50.25	53.59	**68.10**	33.28	37.80	**82.80**	57.58	**55.27**

Table 3: S-NIAH zero-shot accuracy at 1K/2K/4K tokens for 760M models. The multi-key, multi-query, and multi-value experiments have been averaged into a single category for brevity.

	S-NIAH-1			S-NIAH-2			S-NIAH-3			S-NIAH-Multi (avg)			Average		
Model	1K	2K	4K	1K	2K	4K	1K	2K	4K	1K	2K	4K	1K	2K	4K
Transformer	100	100	94.8	100	100	97.6	95.2	81.4	68.2	81.1	62.3	41.1	89.73	78.07	63.98
Mamba2	99.6	99.4	99.2	93.0	99.0	80.4	27.6	41.6	11.4	43.0	27.4	20.9	58.18	53.71	42.26
GateRank+Mamba2	**100**	99.8	90.8	**100**	99.2	52.0	75.6	47.8	21.0	41.0	30.5	21.0	66.44	56.39	37.80
GDNet	**100**	**100**	**99.8**	**100**	99.8	**98.4**	50.2	28.4	17.8	49.2	39.6	27.1	66.28	57.82	49.57
GateRank+GDNet	99.8	99.4	89.4	99.8	**100**	94.0	**91.2**	**78.0**	**44.2**	**61.8**	**43.2**	**29.2**	**79.36**	**67.83**	**52.53**

both linear-attention baselines: at length 1024 it raises accuracy by 69.3 percentage points on Mamba2 (91.2% vs. 21.9%) and by 10.8 percentage points on Gated DeltaNet (84.1% vs. 73.3%); at length 512 both Gated DeltaNet variants are already saturated near 98.5%. The slow state is therefore essential for recall in the longer-sequence regime.

We attribute this gain to two-timescale consolidation: when the fast state resets, locally irrelevant content is cleared while consolidated information is preserved in the slow state. Standard linear attention must instead hold near-unity decay to retain long-range information, causing interfering updates from every token to accumulate in the output. Consistent with this view, the learned slow state update ratio (Table 1) is sparser at length 1024 (0.196) than at length 512 (0.313), indicating that the model consolidates less frequently when more context is available.

The frozen-reset experiment on Compositional MQAR (Figure 4) tests this mechanism directly. Sparser hardcoded schedules outperform dense resets, and the learned schedule approaches the best frozen one. The optimum lies near every third token, matching the task's storage period of one fact per three tokens, supporting the view that resets should track memory events rather than fire at every step.

Finally, Figure 5 isolates higher-rank decay from raw capacity (in a smaller configuration than Figure 3; see Appendix C). GateRank + Mamba2 (size 128 per state) outperforms Mamba2 not only at the matched per-state size, but also at the matched total state size of 256. Simply doubling the baseline's recurrent state does not close the gap, confirming that the gain stems from the more expressive decay structure rather than additional memory.

5.2 Language Modeling, S-NIAH, and LongBench

Table 2 shows that GateRank maintains competitive language modeling performance. GateRank + Gated DeltaNet achieves the best LAMBADA perplexity (16.29) and common sense reasoning average (55.27).

The S-NIAH results (Table 3) show consistent improvements on complex retrieval tasks. GateRank + Gated DeltaNet achieves 79.36% average at 1K tokens versus 66.28% for the baseline, with the largest gains on S-NIAH-3 (91.2% vs 50.2%) and the averaged S-NIAH-Multi tasks (61.8% vs 49.2%). These improvements align with the MQAR findings: breaking the rank-1 decay structure enables more sophisticated memory management for multi-target retrieval.

LongBench-E (Table 4) probes more realistic long-context tasks. With less per-step decode flops than the Transformer (Section F), GateRank substantially improves the LongBench-E average over its linear-attention baselines: a score of 12.76 for GateRank + Gated DeltaNet vs 11.33 for Gated DeltaNet, and 11.34 for GateRank + Mamba2 vs 9.80 for Mamba2, bridging most of the gap to the Transformer (13.40) while retaining the $O(1)$-per-step decode complexity of linear attention.

Table 4: LongBench-E scores (up to 4K tokens) for 760M models. Tasks: 2Wiki, GovRep, Hotpot, LCC, M-News, M-Field, P-Count, Qasper, RepoBench, SamSum, TREC, Trivia.

Model	2W	GR	HP	LC	MN	MF	PC	QS	RB	SS	TR	TQ	Avg
Transformer	9.86	13.70	8.65	23.59	11.30	16.72	8.10	6.56	15.33	23.89	10.28	12.79	13.40
Mamba2	6.90	18.53	6.19	13.39	14.56	12.02	7.26	5.26	13.33	5.96	6.67	7.55	9.80
GateRank+Mamba2	**11.95**	19.59	6.84	**16.97**	**15.76**	12.12	9.25	5.28	18.33	5.43	6.67	7.86	11.34
GDNet	6.33	19.51	6.68	15.68	15.07	11.77	2.27	**6.36**	**20.67**	8.90	**13.33**	9.42	11.33
GateRank+GDNet	11.23	**19.68**	**6.97**	13.55	15.16	**13.59**	**10.36**	5.70	20.33	**16.31**	5.00	**15.28**	**12.76**

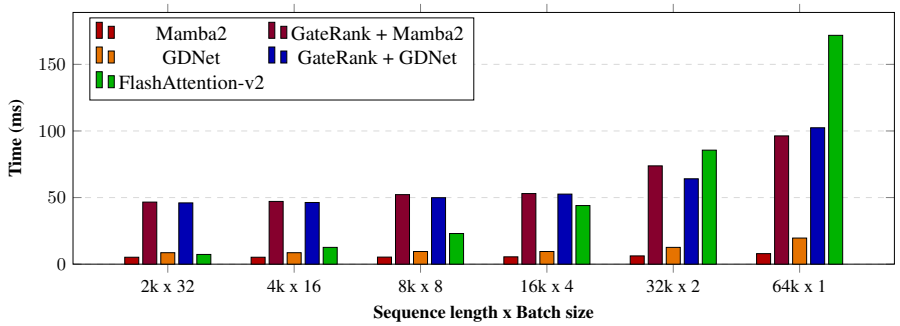

Figure 6: Benchmark results showing time for a forward and backward operation of the attention kernels with dimensions equivalent to the 760M model.

5.3 Throughput

Figure 6 shows that GateRank surpasses FlashAttention-v2 throughput beyond 32K tokens, preserving linear attention's efficiency advantage at long sequences. The overhead over Mamba2 and Gated DeltaNet stems from materializing both fast and slow states at chunk boundaries, which should yield only a $2\times$ slowdown, markedly less than Figure 6 reports.

We attribute the gap to a proof-of-concept Triton kernel that misses software-pipelining opportunities (no overlap of HBM loads with tensor-core updates, no TMA, serial decay computation in the chunk-scan loop). A hand-tuned Hopper/Blackwell kernel should approach the theoretical $2\times$ limit; H100 access constraints prevented us from pursuing this because our RTX 6000 Pro Blackwell GPUs lack relevant features of server-side Blackwell GPUs such as asynchronous tensor core instructions.

Despite this, the structural overhead is small: maintaining two states adds only $\sim$4% per-layer flops at 760M (MLP dominates; see Appendix F), while the parameter-matched Transformer's attention costs $8\times$ more multiplies per decode step at $T = 4096$ and scales as $O(T)$. GateRank thus trades a small constant overhead for recall closer to self-attention without the per-step decode cost.

6 Conclusion

This paper introduces GateRank, a novel extension to linear attention that overcomes fundamental limitations of rank-1 semi-separable decay structures through learnable memory segmentation derived from dual timescale discretization of a continuous time system.

From a practical standpoint, GateRank demonstrates broad applicability as an extension to linear attention architectures while maintaining their efficiency thanks to $O(1)$ decode step complexity and chunk scan prefill kernels. With successful integration into both Mamba2 and Gated DeltaNet, it yields consistent improvements on retrieval-intensive benchmarks.

Looking forward, efficiently alleviating the semi-separable rank-1 structure represents an important direction for improving associative recall in linear attention models while preserving their autoregressive inference advantages stemming from fixed-size memory. GateRank's learnable memory segmentation offers a promising path toward closing the gap with softmax-based self-attention on memory-intensive tasks without incurring self-attention's quadratic computational cost.

Acknowledgments and Disclosure of Funding

References

[1] S. Arora, S. Eyuboglu, A. Timalsina, I. Johnson, M. Poli, J. Zou, A. Rudra, and C. Re. Zoology: Measuring and improving recall in efficient language models. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=LY3ukUANko.

[2] S. Arora, S. Eyuboglu, M. Zhang, A. Timalsina, S. Alberti, D. Zinsley, J. Zou, A. Rudra, and C. Ré. Simple linear attention language models balance the recall-throughput tradeoff. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 1763–1840. PMLR, 2024. URL https://proceedings.mlr.press/v235/arora24a.html.

[3] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. LongBench: A bilingual, multitask benchmark for long context understanding. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar, editors, *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 3119–3137, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.172. URL https://aclanthology.org/2024.acl-long.172/.

[4] S. Biderman, H. Schoelkopf, L. Sutawika, L. Gao, J. Tow, B. Abbasi, et al. Lessons from the trenches on reproducible evaluation of language models. *arXiv preprint arXiv:2405.14782*, 2024.

[5] Y. Bisk, R. Zellers, R. Le Bras, J. Gao, and Y. Choi. PIQA: Reasoning about physical commonsense in natural language. In *Proceedings of the Thirty-Fourth AAAI Conference on Artificial Intelligence*, volume 34.

[6] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei. Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901. Curran Associates, Inc., 2020.

[7] K. Choromanski, V. Likhosherstov, D. Dohan, X. Song, A. Gane, T. Sarlos, P. Hawkins, J. Davis, A. Mohiuddin, L. Kaiser, D. Belanger, L. Colwell, and A. Weller. Rethinking attention with performers. In *Proceedings of the Ninth International Conference on Learning Representations*, 2021.

[8] C. Clark, K. Lee, M.-W. Chang, T. Kwiatkowski, M. Collins, and K. Toutanova. BoolQ: Exploring the surprising difficulty of natural yes/no questions. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 2924–2936, Minneapolis, Minnesota, 2019. Association for Computational Linguistics. doi: 10.18653/v1/N19-1300.

[9] P. Clark, I. Cowhey, O. Etzioni, T. Khot, A. Sabharwal, C. Schoenick, and O. Tafjord. Think you have solved question answering? try ARC, the AI2 reasoning challenge, 2018. URL https://arxiv.org/abs/1803.05457.

[10] T. Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *Proceedings of the Twelfth International Conference on Learning Representations*, Vienna, Austria, 2024.

[11] T. Dao and A. Gu. Transformers are SSMs: Generalized models and efficient algorithms through structured state space duality. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 10041–10071. PMLR, 2024.

[12] L. Gao, J. Tow, B. Abbasi, S. Biderman, S. Black, A. DiPofi, C. Foster, L. Golding, J. Hsu, A. Le Noac'h, H. Li, K. McDonell, N. Muennighoff, C. Ociepa, J. Phang, L. Reynolds, H. Schoelkopf, A. Skowron, L. Sutawika, E. Tang, A. Thite, B. Wang, K. Wang, and A. Zou. The language model evaluation harness, July 2024. URL https://zenodo.org/records/12608602.

[13] A. Gu and T. Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *First Conference on Language Modeling*, 2024. URL https://openreview.net/forum?id=tEYskw1VY2.

[14] A. Gu, K. Goel, and C. Ré. Efficiently modeling long sequences with structured state spaces. In *Proceedings of the Tenth International Conference on Learning Representations*, 2022.

[15] H. Guo, S. Yang, T. Goel, E. P. Xing, T. Dao, and Y. Kim. Log-linear attention. In *Proceedings of the Fourteenth International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=mOJgZWkXKW.

[16] C.-P. Hsieh, S. Sun, S. Kriman, S. Acharya, D. Rekesh, F. Jia, Y. Zhang, and B. Ginsburg. RULER: What's the real context size of your long-context language models? In *Proceedings of the First Conference on Language Modeling*. OpenReview, 2024.

[17] HuggingFace. FineWeb-Edu: A high-quality educational dataset. https://huggingface.co/datasets/HuggingFaceFW/fineweb-edu, 2024. Accessed: 2024.

[18] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 5156–5165. PMLR, 2020. URL https://proceedings.mlr.press/v119/katharopoulos20a.html.

[19] B. Liu, R. Wang, L. Wu, Y. Feng, P. Stone, and Q. Liu. Longhorn: State space models are amortized online learners. In *Proceedings of the Thirteenth International Conference on Learning Representations*, Singapore, 2025.

[20] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *International Conference on Learning Representations*, 2017. URL https://openreview.net/forum?id=Byj72udxe.

[21] T. Mihaylov, P. Clark, T. Khot, and A. Sabharwal. Can a suit of armor conduct electricity? a new dataset for open book question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2381–2391, Brussels, Belgium, 2018. Association for Computational Linguistics. doi: 10.18653/v1/D18-1260.

[22] S. Movahedi, F. Sarnthein, N. Muca Cirone, and A. Orvieto. Fixed-point RNNs: Interpolating from diagonal to dense. In *Advances in Neural Information Processing Systems*, volume 38. Curran Associates, Inc., 2025.

[23] A. Orvieto, S. L. Smith, A. Gu, A. Fernando, C. Gulcehre, R. Pascanu, and S. De. Resurrecting recurrent neural networks for long sequences. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202 of *Proceedings of Machine Learning Research*, pages 26670–26698. PMLR, 2023. URL https://proceedings.mlr.press/v202/orvieto23a.html.

[24] D. Paperno, G. Kruszewski, A. Lazaridou, N. Q. Pham, R. Bernardi, S. Pezzelle, M. Baroni, G. Boleda, and R. Fernández. The LAMBADA dataset: Word prediction requiring a broad discourse context. In *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1525–1534, Berlin, Germany, 2016. Association for Computational Linguistics. doi: 10.18653/v1/P16-1144.

[25] B. Peng, E. Alcaide, Q. Anthony, A. Albalak, S. Arcadinho, S. Biderman, H. Cao, X. Cheng, M. Chung, M. Grella, K. K. GV, X. He, H. Hou, J. Lin, P. Kazienko, J. Kocon, J. Kong, B. Koptyra, H. Lau, K. S. I. Mantri, F. Mom, A. Saito, G. Song, X. Tang, B. Wang, J. S. Wind, S. Wozniak, R. Zhang, Z. Zhang, Q. Zhao, P. Zhou, Q. Zhou, J. Zhu, and R.-J. Zhu.

RWKV: Reinventing RNNs for the transformer era. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 14048–14077, Singapore, 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.936.

[26] B. Peng, D. Goldstein, Q. Anthony, A. Albalak, E. Alcaide, S. Biderman, E. Cheah, X. Du, T. Ferdinan, H. Hou, P. Kazienko, K. K. GV, J. Kocon, B. Koptyra, S. Krishna, R. McClelland Jr., J. Lin, N. Muennighoff, F. Obeid, A. Saito, G. Song, H. Tu, C. Wirawan, S. Wozniak, R. Zhang, B. Zhao, Q. Zhao, P. Zhou, J. Zhu, and R.-J. Zhu. Eagle and finch: RWKV with matrix-valued states and dynamic recurrence. In *Proceedings of the First Conference on Language Modeling*. OpenReview, 2024.

[27] Bo Peng, Ruichong Zhang, Daniel Goldstein, Eric Alcaide, Xingjian Du, Haowen Hou, Jiaju Lin, Jiaxing Liu, Janna Lu, William Merrill, Guangyu Song, Kaifeng Tan, Saiteja Utpala, Nathan Wilce, Johan S. Wind, Tianyi Wu, Daniel Wuttke, and Christian Zhou-Zheng. RWKV-7 "goose" with expressive dynamic state evolution. In *Second Conference on Language Modeling*, 2025. URL `https://openreview.net/forum?id=ayB1PACN5j`.

[28] K. Sakaguchi, R. Le Bras, C. Bhagavatula, and Y. Choi. WinoGrande: An adversarial winograd schema challenge at scale. In *Proceedings of the Thirty-Fourth AAAI Conference on Artificial Intelligence*, volume 34.

[29] I. Schlag, K. Irie, and J. Schmidhuber. Linear transformers are secretly fast weight programmers. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 9355–9366. PMLR, 2021. URL `http://proceedings.mlr.press/v139/schlag21a.html`.

[30] J. Siems, T. Carstensen, A. Zela, F. Hutter, M. Pontil, and R. Grazzi. DeltaProduct: Improving state-tracking in linear RNNs via householder products. In *Advances in Neural Information Processing Systems*, volume 38. Curran Associates, Inc., 2025.

[31] J. T. H. Smith, A. Warrington, and S. W. Linderman. Simplified state space layers for sequence modeling. In *Proceedings of the Eleventh International Conference on Learning Representations*, Kigali, Rwanda, 2023.

[32] Y. Sun, L. Dong, S. Huang, S. Ma, Y. Xia, J. Xue, J. Wang, and F. Wei. Retentive network: A successor to transformer for large language models, 2023. URL `https://arxiv.org/abs/2307.08621`.

[33] P. Tillet, H. T. Kung, and D. Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, MAPL 2019, pages 10–19, Phoenix, AZ, USA, 2019. Association for Computing Machinery. doi: 10.1145/3315508.3329973.

[34] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample. LLaMA: Open and efficient foundation language models, 2023. URL `https://arxiv.org/abs/2302.13971`.

[35] J. Welbl, N. F. Liu, and M. Gardner. Crowdsourcing multiple choice science questions. In *Proceedings of the 3rd Workshop on Noisy User-generated Text*, pages 94–106, Copenhagen, Denmark, 2017. Association for Computational Linguistics. doi: 10.18653/v1/W17-4413.

[36] S. Yang, B. Wang, Y. Shen, R. Panda, and Y. Kim. Gated linear attention transformers with hardware-efficient training. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*. PMLR, 2024.

[37] S. Yang, B. Wang, Y. Zhang, Y. Shen, and Y. Kim. Parallelizing linear transformers with the delta rule over sequence length. In *Advances in Neural Information Processing Systems*, volume 37. Curran Associates, Inc., 2024.

[38] S. Yang, J. Kautz, and A. Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. In *Proceedings of the Thirteenth International Conference on Learning Representations*, Singapore, 2025.

[39] R. Zellers, A. Holtzman, Y. Bisk, A. Farhadi, and Y. Choi. HellaSwag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pages 4791–4800, Florence, Italy, 2019. Association for Computational Linguistics. doi: 10.18653/v1/P19-1472.

[40] M. Zhang, K. Bhatia, H. Kumbong, and C. Ré. The hedgehog & the porcupine: Expressive linear attentions with softmax mimicry. In *Proceedings of the Twelfth International Conference on Learning Representations*, Vienna, Austria, 2024.

[41] Y. Zhang and S. Yang. Flame: Flash language modeling made easy, January 2025. URL `https://github.com/fla-org/flame`.

[42] S. Zhong, M. Xu, T. Ao, and G. Shi. Understanding transformer from the perspective of associative memory, 2025. URL `https://arxiv.org/abs/2505.19488`.

A Implementation details

Λ is computed as the exponential of the output of the function `reverse_reset_cumsum` in listing 2.

Decaying to the last position for the keys requires reversing the cumulative sum in the log-space by taking the exponential of last row of the matrix computed in `reverse_reset_cumsum` from listing 2 which we denote as $\boldsymbol{d}_{\text{last}}$.

The extra fast state multiplier $\boldsymbol{d}_{\text{first}}$ for decaying to the first position in a chunk is computed in the function `same_segment_bwd_mult` in listing 4, using the log-space fast gate g_{fast}.

Lastly, we need the function `chunk_contains_reset` from listing 3 to model the conditional updates of both states at the chunk boundaries.

```
def reset_idcs_cm(g_slow):
    """
    for masking operations in index space
    to avoid floating-point comparisons
    """
    reset_idcs = where(g_slow != 0, arange(g_slow.size(0)), 0)
    reset_idcs_cm = cummax(reset_idcs)
```

Listing 1: PyTorch style reset index computation used for masking

```
def reset_cumsum(g_fast, g_slow):
    """computes the cumsum of g_fast with resets after g_slow != 0"""
    g_fast_cs = cumsum(g_fast)
    # use cummin because gates are always negative in log-space
    reset_vals = where(left_pad(g_slow)[:-1] != 0, left_pad(g_fast_cs)[:-1],
        0)
    lmbda = g_fast_cs - cummin(reset_vals, dim=-1)
    return lmbda

def reverse_reset_cumsum(g_slow, lmbda, reset_idcs_cm):
    """
    computes the mask matrix Lambda for the parallel form
    g_slow: (chunk_size,), lmbda: (chunk_size,)
    returns: (chunk_size,chunk_size)
    """
    last_reset_seg_2d_mask = (lower_diag_mask &
        (F.pad(reset_idcs_cm, (1,0))[:-1] == reset_idcs_cm))
    # perform inverse in log-space with "outer subtraction"
    # for each reset segment; don't subtract at reset locations
    # since local cumsums from previous segment go up to
    # non-zero g_slow indicating start of next segment
    subtrahend = where(g_slow[None,:] == 0, lmbda[None,:], 0)
    seg_inv_2d = lmbda[:,None] - subtrahend
    seg_inv_2d = seg_inv_2d.masked_fill(~last_reset_seg_2d_mask, 0)
    # each row is a sum of previous rows with reset signals in g_slow
    multiplier = (g_slow[None,:] != 0) & lower_diag_mask
    return (identity + multiplier) @ seg_inv_2d
```

Listing 2: PyTorch style reset cumsums and reverse computation with matrix multiplication (see listing 1 for computation of `reset_idcs_cm`)

```
def chunk_contains_reset(g_slow):
    return g_slow.any()
```

Listing 3: Check if there are resets in a chunk given the slow gate vector for the chunk note that a non-zero slow gate indicates a reset in the fast state.

```
def same_segment_bwd_mult(g_slow, lmbda, reset_idcs_cm):
    cond = (left_pad(reset_idcs_cm)[:-1] == 0) & (g_slow[0] == 0)
    masked_offs = where(cond, arange(g_slow.size(0)), 0)
    masked_offs = cummax(masked_offs)
    return gather(lmbda, masked_offs)
```

Listing 4: PyTorch style code for the inter-chunk decay. It uses the `reset_idcs_cm` as computed in listing 1

B Motivation via Dual Timescale Discretization of Continuous-Time System

This section provides a detailed discretization-based motivation for the GateRank discrete-time update equations (Equation 1) using a multi-rate zero-order hold (ZOH) perspective on a continuous-time system.

The continuous-time system is given by:

$$\dot{\mathbf{S}}(t) = \boldsymbol{A}\mathbf{S}(t) + \boldsymbol{B}(t)\boldsymbol{v}(t)^\top \tag{12}$$

The general exact solution to Equation 12 from time t_0 to t is:

$$\mathbf{S}(t) = \exp(\boldsymbol{A}(t-t_0))\mathbf{S}(t_0) + \int_{t_0}^{t} \exp(\boldsymbol{A}(t-\tau))\boldsymbol{B}(\tau)\boldsymbol{v}(\tau)^\top d\tau \tag{13}$$

Traditional sequence models assume a single sampling period. To model fast and slow memory, we introduce a fast sampling period Δ_t^{fast} for input reception and a slow sampling period Δ_t^{slow} for state consolidation, where Δ_t^{slow} is an integer multiple of Δ_t^{fast}.

We evaluate the state transition between two slow updates, setting $t_n = n\Delta_t^{\text{slow}}$ and $t_{n+1} = (n+1)\Delta_t^{\text{slow}}$. Substituting these into the exact solution yields:

$$\mathbf{S}(t_{n+1}) = \exp(\boldsymbol{A}\Delta_t^{\text{slow}})\mathbf{S}(t_n) + \sum_{i=n\frac{\Delta_t^{\text{slow}}}{\Delta_t^{\text{fast}}}}^{(n+1)\frac{\Delta_t^{\text{slow}}}{\Delta_t^{\text{fast}}}-1} \int_{i\Delta_t^{\text{fast}}}^{(i+1)\Delta_t^{\text{fast}}} \exp(\boldsymbol{A}(t_{n+1}-\tau))\boldsymbol{B}(\tau)\boldsymbol{v}(\tau)^\top d\tau \tag{14}$$

We apply the ZOH assumption at the fast timescale, treating $\boldsymbol{B}(\tau)$ and $\boldsymbol{v}(\tau)$ as constant over each interval $[i\Delta_t^{\text{fast}}, (i+1)\Delta_t^{\text{fast}})$, denoted discretely as $\boldsymbol{B}_i$ and $\boldsymbol{v}_i$. The integral over a single fast step simplifies to:

$$\int_{i\Delta_t^{\text{fast}}}^{(i+1)\Delta_t^{\text{fast}}} \exp(\boldsymbol{A}(t_{n+1}-\tau))d\tau = \exp(\boldsymbol{A}(t_{n+1}-i\Delta_t^{\text{fast}}))\boldsymbol{A}^{-1}(\boldsymbol{I} - \exp(-\boldsymbol{A}\Delta_t^{\text{fast}})) \tag{15}$$

To align the indices, we substitute $j = i - nK$, where $K = \frac{\Delta_t^{\text{slow}}}{\Delta_t^{\text{fast}}}$ reprsents the number of terms in the sum. The accumulation term becomes:

$$\begin{aligned}
&\sum_{j=0}^{K-1} \exp\left(\boldsymbol{A}\Delta_t^{\text{fast}}(K-1-j)\right) \exp(\boldsymbol{A}\Delta_t^{\text{fast}})\boldsymbol{A}^{-1}(\boldsymbol{I} - \exp(-\boldsymbol{A}\Delta_t^{\text{fast}}))\boldsymbol{B}_{j+nK}\boldsymbol{v}_{j+nK}^\top \\
&= \sum_{j=0}^{K-1} \exp\left(\boldsymbol{A}\Delta_t^{\text{fast}}(K-1-j)\right) \boldsymbol{A}^{-1}(\exp(\boldsymbol{A}\Delta_t^{\text{fast}}) - \boldsymbol{I})\boldsymbol{B}_{j+nK}\boldsymbol{v}_{j+nK}^\top
\end{aligned} \tag{16}$$

Equation 16 takes the exact form of a discrete convolution $\sum a^{t-j}\boldsymbol{k}_j\boldsymbol{v}_j^\top$. We can isolate the discretized fast-timescale components as:

$$\beta_t = \exp(\boldsymbol{A}\Delta_t^{\text{fast}}) \tag{17}$$

$$\boldsymbol{k}_t = \boldsymbol{A}^{-1}(\exp(\boldsymbol{A}\Delta_t^{\text{fast}}) - \boldsymbol{I})\boldsymbol{B}_t \tag{18}$$

Next, we reformulate Equation 14 by replacing the sum of integrals with the recurrence, yielding the following update during timesteps when the sum is computed:

$$\mathbf{S}_t^{\text{slow}} = \mathbf{S}_{t-1}^{\text{slow}} \qquad \mathbf{S}_t^{\text{fast}} = \beta_t\mathbf{S}_{t-1}^{\text{fast}} + \boldsymbol{k}_t\boldsymbol{v}_t^\top \tag{19}$$

where we used $\alpha_t = \exp(\mathbf{A}\Delta_t^{\text{fast}})$. During the fast steps the slow state is held constant to iteratively compute the sum term of Equation 14.

At the slow update boundary, i.e. after $K = \Delta_t^{\text{slow}}/\Delta_t^{\text{fast}}$ steps, the fast state recurrence starts from a fresh key-value pair while the slow state absorbs the next fast state update:

$$\mathbf{S}_t^{\text{slow}} = \alpha_t \mathbf{S}_{t-1}^{\text{slow}} + \beta_t \mathbf{S}_{t-1}^{\text{fast}} + \boldsymbol{k}_t \boldsymbol{v}_t^\top, \qquad \mathbf{S}_t^{\text{fast}} = 0. \tag{20}$$

Equation 20 is the strict multi-rate ZOH discretization of Equation 12: the slow decay α_t acts only on the previous slow buffer, not on the freshly consolidated chunk, consistent with Equation 14.

Relation to Equation 1. GateRank's main-body recurrence preserves the dual-timescale structure of Equation 20 but applies the slow decay α_t to the freshly consolidated chunk as well, and retains $\boldsymbol{k}_t \boldsymbol{v}_t^\top$ in the fast buffer rather than the slow one:

$$\mathbf{S}_t^{\text{slow}} = \alpha_t \mathbf{S}_{t-1}^{\text{slow}} + \alpha_t \beta_t \mathbf{S}_{t-1}^{\text{fast}}, \qquad \mathbf{S}_t^{\text{fast}} = \boldsymbol{k}_t \boldsymbol{v}_t^\top.$$

Retaining $\boldsymbol{k}_t \boldsymbol{v}_t^\top$ in the fast state at reset positions ensures every key-value pair is subject to the fast decay β_t within its reset segment; the strict-ZOH form would otherwise leave one key-value pair per segment exempt from β_t and governed only by the sparsely-firing, near-identity slow decay α_t. Moreover, the immediate readout operation at the current token remains equivalent because Equation 1c uses the sum of both states. However, this recurrence still deviates from Equation 20 by an extra factor of α_t on the consolidated chunk. The motivation is implementation simplicity: with this choice, the inter-key decay matrix Γ retains the standard linear-attention form $\Gamma_{i,j} = \gamma_i/\gamma_j$ with $\gamma_i = \prod_{j=1}^{i} \alpha_j$, so decay accumulation reuses the cumulative-product machinery of Mamba2-style chunk scans. Strict ZOH, in contrast, would yield $\Gamma'_{i,j} = \gamma_i/\gamma_{r_{\text{next}}(j)}$, where each column index must first be rounded up to its next reset position; this introduces per-column indexing into the chunk-scan kernel that Equation 1 avoids.

Finally, because Equation 1 uses $\alpha_t = \exp(-A_{\text{slow}}\Delta_t^{\text{slow}})$ and $\beta_t = \exp(-A_{\text{fast}}\Delta_t^{\text{fast}})$, where A_{slow}, A_{fast}, $\Delta_t^{\text{slow}} = \text{ReLU}(\text{ShortConv}((W_\alpha x_t + b_\alpha))$, and $\Delta_t^{\text{fast}} = \text{softplus}(W_\beta x_t + b_\beta)$ are learned independently, the extra α_t factor on the consolidated chunk is absorbable into the learned decays, so the two recurrences span the same model class. By further allowing the slow boundaries to be data-driven and dynamic rather than strictly periodic, and by decoupling the two timescales through separate learned scalars A_{slow} and A_{fast}, we recover the dynamics of GateRank presented in Section 3.

C MQAR Experimental Details

Task Setup The MQAR task presents sequences of key-value pairs $(k_1, v_1, k_2, v_2, \ldots, k_m, v_m)$ followed by query tokens. For each query k_i, the model must predict v_i. We use vocabulary size 8192 and pair sequence length 512 with $m = 64$ key-value pairs and sequence length 1024 with $m = 128$ key-value pairs in Figure 3; Figure 5 only uses sequence length 512 and sweeps $m \in \{4, 16, 24, 32, 40, 48, 56, 64\}$.

For Compositional MQAR (used in Figure 4), sequences have the form $(k_{1,1}, k_{1,2}, v_1, k_{2,1}, k_{2,2}, v_2, \ldots)$, requiring two-hop retrieval: the model must first retrieve the intermediate key $k_{i,2}$ given $k_{i,1}$, then use $k_{i,2}$ to retrieve the final value v_i. We use $m = 36$ pairs at sequence length 512.

Shared training settings All MQAR experiments use $d_{\text{model}} = 64$ and two layers, a short convolution of kernel size 4, 100,000 training examples and 3,000 test examples per configuration, and sweep learning rates over 4 log-spaced values from 10^{-4} to 10^{-2} across 4 random seeds. All experiments were performed on 4 RTX 3090 GPUs and took between 2 and 24 hours.

For a fair comparison, the scaling parameterization used in the exponential gating mechanism is aligned between the baseline models and GateRank. While the baseline models dictate their decay through a softplus activation via $\exp(-A \cdot \text{softplus}(\cdot))$, GateRank's slow state utilizes a similarly scaled ReLU activation to determine g_{slow} through $\log(\alpha_t) = -A_{\text{slow}} \cdot \text{ReLU}(\cdot)$, as described in Section 3. Sharing this scalar parameterization scheme improved associative recall performance for the baselines and ensures a fair structural comparison with GateRank's slow state memory behavior.

Figure 3 setup

- **(GateRank +) Mamba2:** 2 heads of head dimension 64, expansion factor 2, state size 64 (matched to GateRank's per-state size of 64), trained for 128 epochs. GateRank + Mamba2 uses the same architecture as Mamba2 with the addition of the slow state and reset gate.
- **(GateRank +) Gated DeltaNet:** 2 heads of head dimension 32 with value-expansion factor 4, giving the same per-head state of $32 \times 128 = 4{,}096$ as Mamba2 at head dimension 64 and state size 64; trained for 128 epochs. The use of the output gate is swept on/off for both variants, and the plot picks the best per-seed accuracy across this choice and across the learning-rate sweep.
- **Transformer (parameter-matched reference):** 2 heads with a depthwise convolution of kernel size 4, trained for 64 epochs.

Figure 4 setup GateRank + Mamba2 only, on Compositional MQAR with 2 heads of head dimension 64, expansion factor 2, state size 128, trained for 64 epochs. Four reset patterns are compared: *Dense* ($g_{\text{slow}} = 0.01$ at every position), *Alternating* ($g_{\text{slow}} = 0.01$ at odd positions, 0 at even positions), *Every Third* ($g_{\text{slow}} = 0.01$ where pos $\bmod 3 = 2$, 0 elsewhere), and *Learned* (g_{slow} computed normally from the network). The small fixed value $g_{\text{slow}} = 0.01$ is used for the frozen patterns to minimise forgetting in the slow state.

Figure 5 setup GateRank + Mamba2 with fast and slow state of size 128 each, versus Mamba2 baselines at state sizes 128 and 256. To reduce the runtime of this larger sweep (8 kv values $\times$ 4 seeds $\times$ 4 learning rates per model) while still surfacing clearer differences between the curves, the configuration is intentionally smaller than that of Figure 3: only sequence length 512 (no 1024), 64 training epochs (a value also commonly used in the Zoology MQAR repository), and a single head per layer of head dimension 128 (instead of 2 heads of head dimension 64). The remaining settings (expansion factor 2 and the same short convolution as in Figure 3) match the Figure 3 Mamba2 family.

D Training Details

760M Model Configuration All models use dimension 1536. Mamba2-based models have 20 layers, Gated DeltaNet-based models have 18 layers, and the Transformer baseline has 24 layers (all adjusted for parameter parity). We train on 30B tokens from FineWeb-Edu with batch size 120, sequence length 4096, AdamW optimizer, learning rate 1.25e-3 with cosine decay, weight decay 0.1, and gradient clipping at norm 1.0. Training uses the Flame framework [41] across 8 RTX 6000 Pro Blackwell GPUs and took around 2 days for one model.

Evaluation Benchmarks Language modeling: WikiText [20], LAMBADA [24]. Common sense reasoning: PIQA [5], HellaSwag [39], WinoGrande [28], ARC [9], OBQA [21], SCIQ [35], BoolQ [8]. Retrieval: S-NIAH from RULER [16] at 1K, 2K, 4K tokens. Realistic long-context tasks: LongBench-E [3] at 4K tokens.

E 760M Model Reset Statistics

To better understand the learned reset dynamics at scale, we evaluated the reset statistics of the 760M-parameter GateRank + Gated DeltaNet model during the prefill of 5 WikiText sequences. The per-layer statistics and head-level analysis are summarized below, with layer-wise aggregated data provided in Table 5.

Our analysis of the reset behavior reveals the following main findings:

- **Early layers reset aggressively:** The initial layers (layers 0–1) exhibit very high reset rates ($\sim 97\%$).
- **Middle and later layers show high diversity:** As reflected by the high standard deviations in Table 5, heads within a single layer behave very differently. One head may reset at nearly 100%, while another resets at $< 1\%$. This suggests strong head specialization into different memory timescales.

Layer	Mean Reset %	Std
0	97.7%	2.3%
1	96.7%	3.3%
2	76.7%	33.2%
3	68.3%	39.9%
4	53.7%	41.0%
5	57.1%	36.6%
6	70.5%	29.8%
7	43.5%	31.3%
8	48.0%	28.8%
9	50.1%	37.2%
10	47.3%	33.1%
11	54.9%	35.5%
12	47.4%	30.1%
13	24.7%	31.8%
14	58.8%	35.4%
15	32.2%	32.5%
16	37.4%	37.1%
17	48.7%	39.9%

Table 5: Mean Reset Percentage and Standard Deviation per Layer for the 760M GateRank + Gated DeltaNet model.

- **Non-reset segment lengths vary enormously:** Frequent resetters have median non-reset segments of just 1–2 tokens, while sparse resetters reach median lengths of 10–30 tokens, with maximum segments extending from 200 to 2000+ tokens.
- **A few heads act as near-permanent memory:** Certain heads demonstrate reset rates of $< 1\%$, with memory segments reaching the full sequence length of 2048 tokens. These heads rely entirely on the fast state for any forgetting dynamics.

Overall, these results indicate that GateRank effectively learns to represent token interactions across various timescales by specializing the memory consolidation patterns of individual heads and layers.

F Per-Layer Flop Budget

GateRank adds only $\sim$4% flops per layer over the Mamba2 baseline. GateRank's extra cost beyond Mamba2 is one additional state update and readout per head per layer. We count only the major operations that differ using the dimensions of the 760M models: the outer-product update $k_t v_t^\top$ ($d \times n = 64 \times 128 = 8{,}192$ multiplies) and the readout $q_t^\top S_t$ ($d \times n = 8{,}192$ multiplies), giving $2 \times 8{,}192 = 16{,}384$ extra multiplies per head. Across all 48 heads, the extra state cost is $48 \times 16{,}384 = 786{,}432$ multiplies per layer. The SwiGLU MLP, shared identically between both models, costs $3 \times d_{\text{int}} \times d_{\text{model}} = 3 \times 4096 \times 1536 = 18{,}874{,}368$ multiplies (up-projection, gate, and down-projection). Including the Mamba2 baseline state ops of $48 \times 16{,}384 = 786{,}432$, the total Mamba2 layer cost is $786{,}432 + 18{,}874{,}368 = 19{,}660{,}800$, and the GateRank layer cost is $1{,}572{,}864 + 18{,}874{,}368 = 20{,}447{,}232$, an increase of $786{,}432/19{,}660{,}800 = 4.0\%$.

Long-context performance scales with attention/state-update compute, and GateRank is a compute-efficient way to improve it. In autoregressive decoding, each KV-cache entry or recurrent-state scalar must be touched from memory exactly once per step. Since more compute per step means the model can store and retrieve more information per layer, there is a direct link between decode flops and long-context/recall capability, and indeed self-attention consistently outperforms linear attention on recall tasks.

Self-attention's decode cost is far higher than linear attention models including GateRank. For the parameter-matched Transformer ($d_{\text{model}} = 1536$, 24 heads, $d_k = d_v = 64$), attention at sequence length $T = 4096$ costs $24 \times T \times (d_k + d_v) = 24 \times 4096 \times 128 = 12{,}582{,}912$ multiplies per layer, while GateRank's dual-state recurrence costs $48 \times 2 \times 2 \times 64 \times 128 = 1{,}572{,}864$, 8$\times$ less. Moreover, this gap grows linearly with sequence length since attention scales as $O(T)$ per decode step while the recurrence is $O(1)$.